

可见光终端客户端通信协议说明

时间：2021/ 09/ 20

声明:

- 请仔细阅读文档，我司对用户不当操作或其他程序导致造成的数据丢失不承担任何责任。
- 本手册是在 **sdk 2.0** 的基础上设计的

目录

目录	3
1 . SDK 描述介绍	1
1.1 本文术语解释	2
2 . 机器与 web 服务器的通讯连接	2
2.1 机器与 Web 服务器的通讯流程分为两种	3
2.2 采用 HTTP 通讯协议	3
2.3 web 服务器下发指令到机器的执行流程	4
2.4 HTTP 请求和应答中所使用的二进制数据的格式	7
2.5 考勤机实现接收服务器指令所需的请求和应答	7
2.6 服务器的应答对于接收指令的请求	9
2.7 机器上传指令执行结果所需的请求和答应	11
2.8 服务器的应答对于机器上传结果的请求	12
3 . 常用指令	13
3.1 服务器端常用指令。	13
3.2 常用指令机器与服务器请求和应答说明	14
3.2.1 设备心跳	14

3.2.2 GET_DEVICE_INFO (获取设备基本信息)	15
3.2.3 GET_USER_ID_LIST(获得注册人员列表)	18
3.2.4 SET_USER_INFO(向考勤机同步写入用户信息)	20
3.2.5 GET_USER_INFO(从考勤机获取用户登记信息)	23
3.2.6 DELETE_USER(删除人员)	25
3.2.7 GET_LOG_DATA (获取记录数据)	28
3.2.8 CLEAR_LOG_DATA (删除所有记录数据)	30
3.2.9 SET_TIME (同步时间)	32
3.2.10 RESET_FK (机器重新启动)	33
3.2.11 UPDATE_FIRMWARE (更新固件)	34
3.2.12 CLEAR_MANAGER(清除设备管理员)	35
3.2.13 ENTER_ENROLL (远程进入注册界面)	36
3.2.14 SET_DEVICE_SETTING(设置设备参数)	37
3.2.15 GET_DEVICE_SETTING(获取设备参数)	41
3.2.16 RESET_DEVICE (恢复出厂设置)	43
3.2.17 SET_DOOR_STATUS (设置门控继电器状态)	44
3.2.18 RESET_ENROLL_MARK (重置实时登记数据标志)	45
3.2.19 RESET_LOG_MARK (重置实时打卡数据标志)	46
4 . 实时数据传输功能	50
4.1 实时打卡记录传输	51
4.2 实时门状态	52
4.3 实时登记数据传输	53
5 . 版本更新记录	55

1 . 描述介绍

在线通信客户端 **API** 是一个用于 互联网云端和在线的考勤机门禁设备进行数据通信的接口 。可以更加方便的管理多台考勤门禁设备， 管理用户信息和指纹，下载考勤记录，后台操作记录，修改用户信息、门禁时间、设置设备和配置访问控制。
设备离线后会将考勤登记数据保存，待在线后会将数据上传到云服务器，

这个 **API** 包含:

1. 下载考勤记录。
2. 上传和下载用户信息、卡信息和指纹信息.人脸信息.掌纹信息。
3. 设置访问控制设备的访问控制规则。
4. 查看设备信息，固件版本，设备考勤记录数，可登记人脸和指纹数量,识别方式等。
5. 实时查看设备的各种事件，例如指纹验证，人员打卡，人员登记，门禁状态，人脸验证。
6. 直接在线注册用户，在线修改用户信息。
7. 设置用户通过门禁的时间段和权限。

1.1 本文术语解释

名词:	解释
机器	互联网上与 HTTP 服务器通讯的考勤/门禁设备
平台管理员	服务器端对特定机器发送指令的人，平台管理员在 PC 上通过 WEB 应用向机器发送指令而且等待其执行结果
下发指令	操作者向机器发布的指令。 例如:设置机器的时间或则更改机器上注册的用户名等都是操作者指令的一个实例。
人员登记数据	机器上用于验证识别用户的数据。例如密码、ID 卡号、指纹、人脸、掌纹等数据。

2 . 机器与 web 服务器的通讯连接

首先，1.成功部署 demo。机器连接到网络。在连接过程中，设备可以被视为一台独立的 PC 机。设备须连接网络才能进行数据通信。

2 设备的服务器 IP 和端口必须与要连接的服务器网页 IP 地址和端口匹配。在不同的连接过程中，您需要设置不同的设备网络，如广域网，选择无线网络,以太网连接等。否则，连接可能会失败。有时，如果设备串行端口繁忙，您可以重新启动设备。

3 连接成功后。web 操作员可以根据机器名或机器 ID 向机器发送命令。

2.1 机器与 Web 服务器的通讯流程分为两种

机器与 Web 服务器的通讯流程的细节可以参考 图 2-1.指令处理过程中的通讯流程

机器和服务器的通信主要按两种模式分类。

- 1 服务器下发指令，机器接收并执行平台管理员下发指令的流程
- 2.机器通知服务器有事件发生，服务器进行响应(例如:用户登记数据发生变化或者有新的记录产生)的流程。

2.2 采用 HTTP 通讯协议

说明：通讯协议指双方实体完成通信或服务所必须遵循的规则和约定。协议定义了数据单元使用的格式，信息单元应该包含的信息与含义，连接方式，信息发送和接收的时序，从而确保网络中数据顺利地传送到确定的地方。

HTTP 通讯方式：

机器向服务器发送的所有的请求用 POST 方式的 HTTP 请求。Body 部分内容全部为 json 格式，其中的二进制内容全部由 base64 组成

2.3 web 服务器下发指令到机器的执行流程

处理过程如下：

- 1 第一步 平台管理员选择考勤机，获得机器 ID（`device_id`）。
- 2.第二步 平台管理员在 WEB 服务器联动的数据库上保存该机器所需执行的指令。
- 3 第三步 机器每隔一定时间向服务器询问有没有针对自己发送的指令，如果有就拿过来执行，并且将其结果上传到服务器。
- 4 第四步 平台管理员每隔一定时间询问服务指令的执行状态。如果有已执行的标识，就处理该结果。
这里 `trans_id` 是任务识别号，即返回指令执行结果时用这个识别号来判断这个结果是对应哪一条指令的。

这个流程表示如图 2-1。

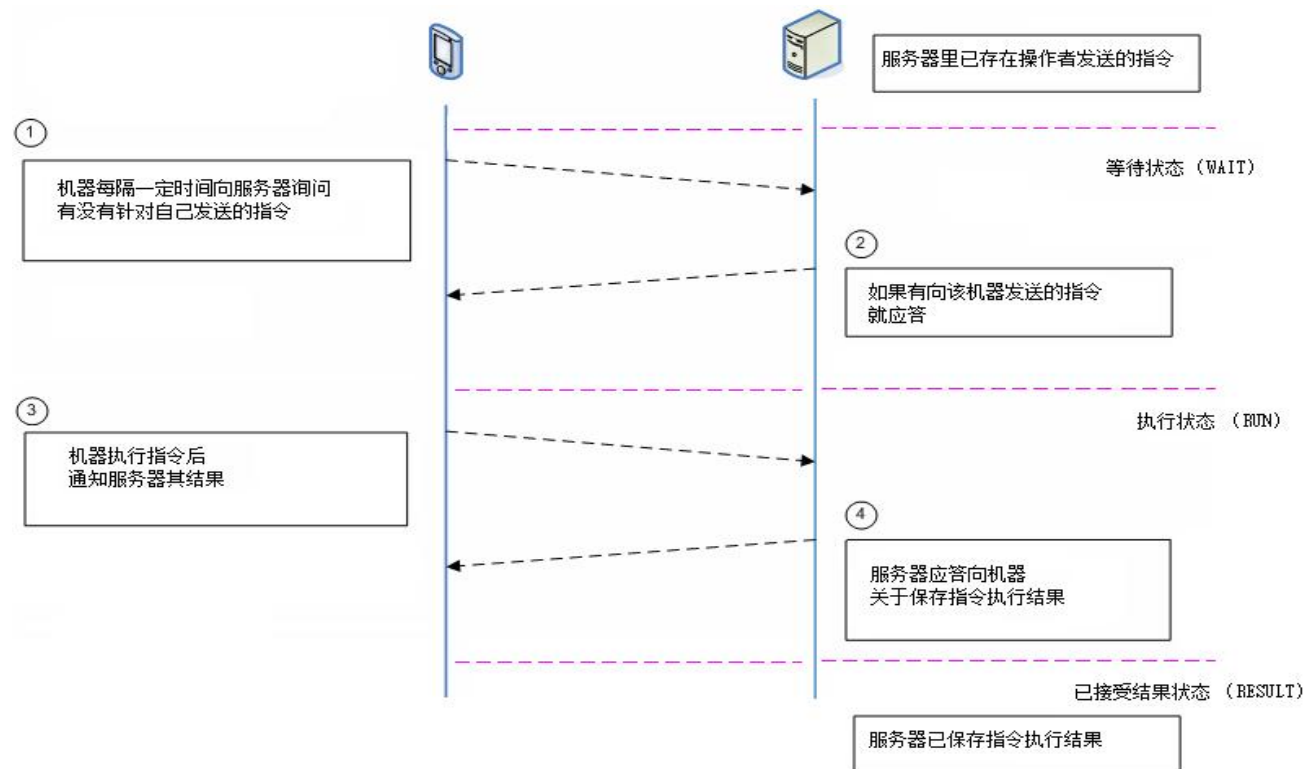


图 2-1.指令处理过程中的通讯流程

若设备需要上传的数据包很大，可能存在分包传输，传输流程如图 2-2 所示，详细过程见指令

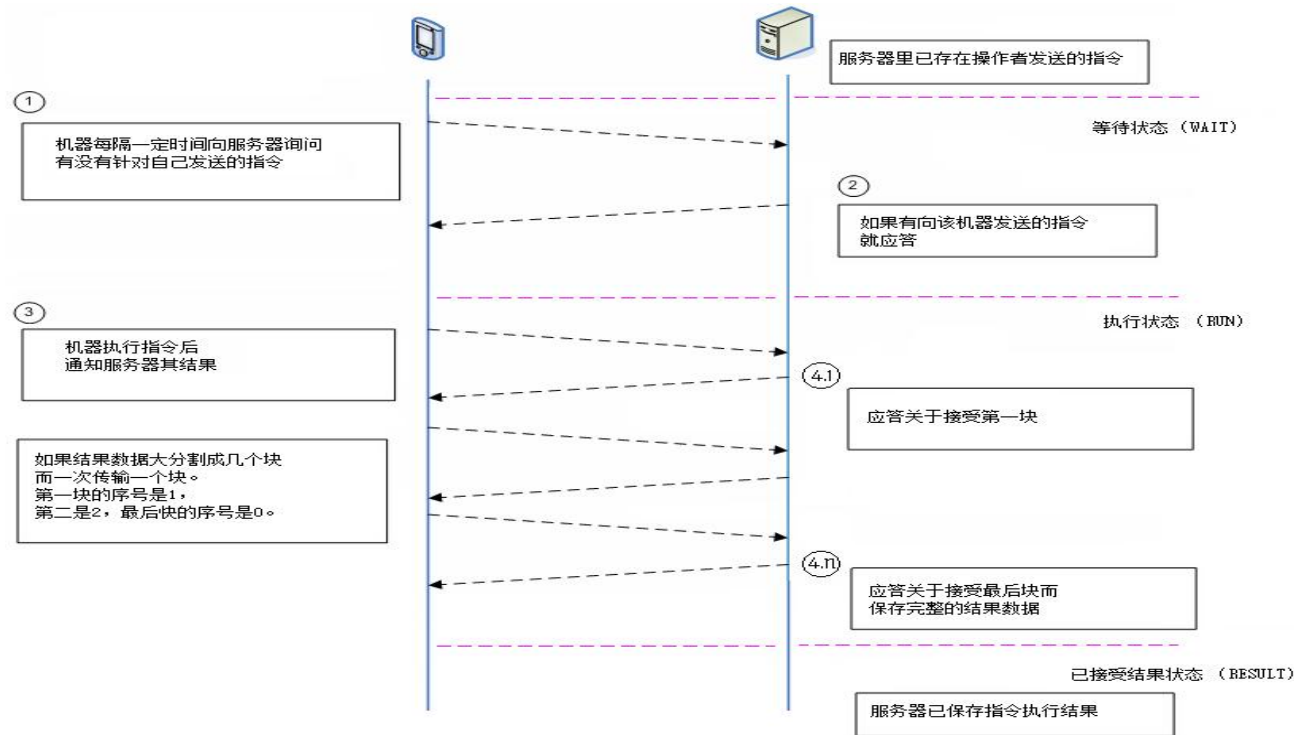


图 2-2.指令处理过程中的通讯流程

第一到二步 其记录中包含如下信息。

任务识别号 (trans_id)，
机器识别号 (device_id)，
指令识别号 (cmd_code)，
指令参数数据 (cmd_param)，

任务状态（trans_status），
任务状态最后更新时间（trans_status_update_time）

2.4 HTTP 请求和应答中所使用的二进制数据的格式

所有二进制数据全部采用 base64 编码放置在 json 格式中。

2.5 考勤机实现接收服务器指令所需的请求和应答

为了接收服务器向自己发送的指令，考勤机每隔一定时间向 WEB 服务器发送 Http request，并接收 response，具体格式如下：

1. 机器接收 web 服务器 下发指令的请求

机器为了接收向自己发送的指令，考勤机每隔一定时间向 WEB 服务器发送 HTTP POST 请求。
这时 HTTP 头部（header）里放置如下字段。

字段名	字段含义	是否必填项	限制说明	参数说明
request_code	需求代码	是	必须是如下字符串。 "receive_cmd"	表示机器向服务器询问有没有对自己发送的指令。
dev_id	机器识别号	是	英数文字，最大 24 位。	连接到同一个 WEB 服务器的所有考勤机，必须有唯一的识别号码。 此参数指考勤机的唯一的号码。

dev_model	用于区分设备类型	是	英文数字 最大 16 位	设备类型，根据不同的类型，设备所使用的协议可能不同
token	设备通行令牌	是	MD5(dev_id + 密钥)	厂商将给每个不同的开发商下发一个密钥，对应的设备种将密钥与设备唯一识别码做字符串拼接的MD5签名，开发商在服务端可以效验该签名的准确性来区分设备是否合法

例如:HTTP 头从机器被上传到服务器:

POST / HTTP/1.0

Accept: image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, application/vnd.ms-excel, application/msword, application/vnd.ms-powerpoint, */*

Accept-Language: en-us

Accept-Encoding: gzip, deflate

User-Agent: Mozilla/4.0

Connection: close

Content-Length: 201

Content-Type: application/octet-stream

request_code: receive_cmd

dev_id: C2689C470326192F

dev_model: Y80X

token: EE5F15B6AF14992F1D9E741397DA6A72

上面一 BOLD 字体来标记的是该注意的字段。

HTTP body 部分里放置 在 1.3 描述的数据。

Body 部分放置的字符串的内容是如下。

```
{
  "time":<1>,
}
```

详见 3.2.1 设备心跳介绍

机器提交接收指令的 HTTP 请求时在 HTTP body 部分不放置任何二进制数据。

2.6 服务器的应答对于接收指令的请求

如果服务器接收上述的请求就查询有没有对该机器发送的指令而如有的话下传应答。

应答的头 (header) 和体 (body) 里所包含的内容是如下。

应答的头(header) 里包含如下字段。

字段名	字段	是否必填选项	限制	参数说明
response_code	应答代码	是	英数文字, 最大 64 位。 英文字母都是大写。	表示接收指令是否成功。 OK: 成功 ERROR: 失败 ERROR_NO_CMD: 没有指令需要执行, 详见 3.2.1
trans_id	任务识别号	否	英数文字, 最大 16 位。	跟踪指令执行过程的识别号码。
cmd_code	指令识别号	否	英数文字, 最大 32 位。 英文字母都是大写。	表示指令类型的字符串。 如: GET_ENROLL_DATA

实例) 如下应答被下传。

```
HTTP/1.1 200 OK  
Cache-Control: private  
Server: Microsoft-IIS/7.5  
Set-Cookie: ASP.NET_SessionId=453lmc45jaelft45glb2mdre; path=/; HttpOnly  
X-AspNet-Version: 2.0.50727  
X-Powered-By: ASP.NET  
Date: Wed, 10 Dec 2014 04:47:42 GMT  
Connection: close  
Content-Length: 39  
Content-Type: application/octet-stream  
response_code: OK  
trans_id: 201  
cmd_code: GET_ENROLL_DATA
```

应答体 (body) 部分里放置的数据按指令不同。

2.7 机器上传指令执行结果所需的请求和答应

机器执行指令后，发送 HTTP request，接收 response，将其结果告诉 WEB 服务器，具体格式如下：

1. 机器上传操作者指令执行结果所需的请求

机器上传到服务器如下的 HTTP POST 请求。

此时请求的头部放置如下字段。

字段名	字段含义	是否必填选项	限制	详细说明
request_code	需求代码	是	必须是如下字符串。 "send_cmd_result"	表示机器向服务器上传指令执行结果。
dev_id	机器识别号	是	英数文字，最大 24 位	参考 2.1.1
dev_model	区分设备类型	是	英文数字，最大 16 位	参考 2.1.1
token	设备通行令牌	是	MD5(dev_id + 密钥)	参考 2.1.1
trans_id	任务识别号	是	英数文字，最大 16 位	参考 2.1.2
cmd_return_code	指令结果代码	是	英数文字，最大 64 位	表示机器执行指令的结果是如何。 执行成功时 "OK"。 执行过程中发生意外 "ERROR" 字符串或则后续说明意外的字符串

此请求的体（body）里放置的数据按指令不同。

2.8 服务器的应答对于机器上传结果的请求

服务器保存数据库机器上传的指令执行结果而下传如下应答。
应答头部放置如下字段。

字段名	字段含义	必要选项	限制	详细说明
response_code	应答代码	是	英数文字，最大 64 位。 英文字母都是大字。	表示服务器成功接收并保存结果数据。 OK：成功 ERROR：失败
trans_id	任务识别号	是	数字，最大 16 位。	参考 2.1.2

3 . 常用指令

3.1 服务器端常用指令。

命令代码	命令名称
GET_DEVICE_INFO	获取设备基本信息
GET_USER_ID_LIST	获取用户 ID 列表
SET_USER_INFO	设置用户信息
GET_USER_INFO	获取用户信息
DELETE_USER	删除用户
GET_LOG_DATA	按时间获取机器上记录
CLEAR_LOG_DATA	清除设备上所有打卡记录
SET_TIME	同步设备时间
RESET_FK	重启设备
UPDATE_FIRMWARE	升级设备固件
CLEAN_MANAGER	一键清除设备管理员
ENTER_ENROLL	进入注册状态
SET_DEVICE_SETTING	设置设备参数
GET_DEVICE_SETTING	获取设备参数

3.2 常用指令机器与服务器请求和应答说明

3.2.1 设备心跳

获得该机器里保存的各种指令。

- 设备请求(之后所有服务端指令都是从设备发起心跳请求开始的，头部部分内容省略，详见 2.5)

-- HTTP header --

request_code: receive_cmd

dev_id: C2689C470326192F

dev_model: Y80X

token: EE5F15B6AF14992F1D9E741397DA6A72

-- HTTP body --

{

"time": "201910010645"

}

- 服务端应答(若服务端没有任何指令需要下发给设备，则发送给设备 **ERROR_NO_CMD** 应答，此应答没有 **body** 部分，部分头部内容省略，详见 2.6)

-- HTTP header --

response_code: ERROR_NO_CMD

trans_id:

cmd_code:

● 参数解释

字段	必须项	说明
time	是	设备发送请求时候的时间

3.2.2 GET_DEVICE_INFO（获取设备基本信息）

用途： 获取设备的基本信息，用于之后的数据处理。服务端应该在设备初次连接的时候自动去获取一次设备的基本信息。

● 设备心跳(见 3.2.1)

● 服务端应答(若服务端存在指令下发，则 **cmd_code** 头填入指令码，根据指令内容决定是否存在 **body**，本条指令无 **body** 部分)

```
-- HTTP header --
response_code:OK
trans_id:100
cmd_code: GET_DEVICE_INFO
```

● 设备应答

```
-- HTTP header --
request_code: send_cmd_result
trans_id:100
cmd_return_code: OK
```

```
-- HTTP body --  
{  
  "name":"设备名",  
  "deviceId":"设备 ID",  
  "firmware":"固件名",  
  "fpVer":"FP_128",  
  "faceVer":"FACE_D_100",  
  "pvVer":"PV_100",  
  "maxBufferLen":92160,  
  "userLimit":3000,  
  "fpLimit":10000,  
  "faceLimit":3000,  
  "pvLimit":3000,  
  "pwdLimit":3000,  
  "cardLimit":3000,  
  "logLimit":200000,  
  "userCount":1234,  
  "managerCount":2,  
  "fpCount":1111,  
  "faceCount":123,  
  "pvCount":123,  
  "pwdCount":456,  
  "cardCount":789,  
  "logCount":1234,  
  "allLogCount":6666,
```

```

"supportACL":1,
"supportShift":0

}

```

- 服务端确认(后续服务端对 `send_cmd_result` 的回答都为这种模式，没有 `body` 部分，头部部分内容省略，详见 2.8)

```

-- HTTP header --
response_code:OK
trans_id:100

```

- 参数解释

字段	必须项	说明
name	是	设备名
firmware	是	固件名
fpVer	否	指纹数据版本，用于各种型号的机器的指纹数据之间的转换。
faceVer	否	人脸数据版本，用于各种型号的机器的人脸数据之间的转换。
pvVer	否	掌纹数据版本，用于各种型号的机器的掌纹数据之间的转换。
maxBufferLen	是	最大数据缓存，设备所接受的一包数据的最大大小，服务端所发送的 <code>httpbody</code> 中 <code>json</code> 长度应该要小于这个值。
userLimit	是	最大用户数
fpLimit	否	最大指纹数
faceLimit	否	最大人脸数
pvLimit	否	最大掌纹数
pwdLimit	否	最大密码数
cardLimit	否	最大卡数
logLimit	是	最大记录数

userCount	是	当前用户数
managerCount,	是	当前管理员数
fpCount	否	当前指纹数
faceCount	否	当前人脸数
pvCount	否	当前掌纹数
pwdCount	否	当前密码数
cardCount	否	当前卡数
logCount	是	当前记录数
allLogCount	是	总记录数
supportACL	否	是否支持专业门禁
supportShift	否	是否支持排班

3.2.3 GET_USER_ID_LIST(获得注册人员列表)

用途： 获取注册在机器里面的用户 ID 列表

- 设备心跳(见 3.2.1)

- 服务端应答

-- HTTP header --

response_code:OK

trans_id:100

cmd_code: GET_USER_ID_LIST

● 设备应答

-- HTTP header --

request_code: send_cmd_result

trans_id:100

cmd_return_code:OK

-- HTTP body --

{

 "packagelId":1,

 "usersCount":1234,

 "usersId":["1","2","3",...]

}

● 服务端确认(详见 2.8)

-- HTTP header --

response_code:OK

trans_id:100

● 重复设备应答和服务端确认流程直到 package_id 的值为 0 即数据传输完成

● 参数解释

字段	必须项	说明
packagel d	是	数据包 ID, 需要上传的数据过大的情况下, 设备将会对数据进行分包传输, 数据包 ID 从 1 开始计数, 依次递增, 0 表示数据全部传输完成
usersCou nt	是	用户 ID 数

userId	否	用户 ID 列表
--------	---	----------

3.2.4 SET_USER_INFO(向考勤机同步写入用户信息)

用途：把对应 ID 的指纹、脸部、掌纹、密码、卡号、名称、权限,用户照片等数据写入对应设备里面。

- 设备心跳(见 3.2.1)

- 服务端应答

```
-- HTTP header --
response_code:OK
trans_id:100
cmd_code: SET_USER_INFO
```

```
-- HTTP body --
{
  "users":[{
    "userId":"1234",
    "name":"张三",
    "privilege":0,
    "card":"123456",
    "pwd":"666",
    "fps":["base64_0"," base64_1",...," base64_9"],
    "face":"base64",
    "palm":["base64_0"," base64_1"],
```



```
"photo":"base64",
"vaildStart":"yyyyMMdd",
"vaildEnd":"yyyyMMdd",
"timeGroups":{
  "sun":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
  "mon":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
  "tue":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
  "wed":["HHmmHHmm","HHmmHHmm","HHmmHHmm"],
  "thu":["HHmmHHmm","HHmmHHmm"],
  "fri":["HHmmHHmm"]
},
"update":1,
"photoEnroll":1
},...]
}
```

- 设备应答

```
-- HTTP header --
request_code: send_cmd_result
trans_id:100
cmd_return_code:OK

-- HTTP body --
{
  "userId":["1","2","3"...]
}
```

● 服务端确认(详见 2.8)

```
-- HTTP header --
response_code:OK
trans_id:100
```

● 参数解释

字段	必须项	说明
users	是	下发的用户数组，注意根据设备的最大 buffer 确定每次下发的用户个数
userId	是	用户 ID
name	否	用户名
privilege	否	用户权限，0 表示普通用户，1 表示管理员
card	否	卡号
pwd	否	密码
fps	否	指纹数据数组，二进制数据，用 base64 表示
face	否	人脸数据，二进制数据，用 base64 表示
palm	否	掌纹数据数组，二进制数据，用 base64 表示
photo	否	用户照片，二进制数据，用 base64 表示
vaildStart	否	用户有效期起始日期
vaildEnd	否	用户有效期结束日期
timeGroups	否	用户的门禁时间段，默认全天全时段有效，详细内容见 3.2.20
update	否	0:原 ID 的相关数据将被删除，用下发的数据取代 1:原 ID 的先关数据将被保存，只覆盖相对应的下发数据 若本项不存在，默认为 0
photoEnroll	否	1:使用照片注册人脸;0:不使用。若本项不存在默认为 0 若此项为 1 则将忽略下发的 face 数据，并删除设备内的原人脸数据，使用下发的照片注册人脸。
usersId	否	下发失败的用户 ID

3.2.5 GET_USER_INFO(从考勤机获取用户登记信息)

用途：从对应考勤机获取用户的登记信息如:指纹、脸部、掌纹、密码、ID 卡、姓名、照片、权限等。因为获取的数据量可能很大，数据需要分包发送(流程图见 2.3 的图 2-2)。注意下发的用户 ID 有在设备内不存在的，则返回的用户数将会小于请求的 ID 数组大小。

- 设备心跳(见 3.2.1)

- 服务端应答

-- HTTP header --

response_code:OK

trans_id:100

cmd_code: GET_USER_INFO

-- HTTP body --(userId 可为空，但是 body 项不可为空，即可以为一个空 json 对象{})

```
{  
  "userId":["1","2","3"...]  
}
```

- 设备应答

-- HTTP header --

request_code: send_cmd_result

trans_id:100

cmd_return_code:OK

```
-- HTTP body --
{
  "packageld":1,
  "usersCount":5
  "users":[ {
    "userId":"1",
    "name":"张三",
    "privilege":0
    "card":"123456"
    "pwd":"666"
    "fps":["base64_0"," base64_1",...," base64_9"],
    "face":"base64",
    "palm":["base64_0"," base64_1"],
    "photo":"base64",
    "vaildStart":"yyyyMMdd",
    "vaildEnd":"yyyyMMdd",
    "timeGroups":{
      "sun":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
      "mon":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
      "tue":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
      "wed":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
      "thu":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
      "fri":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
      "sat":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"]
    }
  }
}
```

```
    },...]
}
```

● 服务端确认(详见 2.8)

```
-- HTTP header --
response_code:OK
trans_id:100
```

● 重复设备应答和服务端确认流程直到 package_id 的值为 0 即数据传输完成

● 参数解释

字段	必须项	说明
userId	否	想要获取的用户 ID 数组，若此项不存在，则视为获取所有用户数据。
packageId	是	数据包 ID，需要上传的数据过大的情况下，设备将会对数据进行分包传输，数据包 ID 从 1 开始计数，依次递增，0 表示数据全部传输完成
usersCount	是	本包数据中包含的用户数量
users	是	设备应答的用户数组，用户结构同 3.2.4 的 users。

3.2.6 DELETE_USER(删除人员)

用途：从设备删除相应 ID 的登记信息。对于某些设备型号，删除某用户的用户信息将会同时删除该用户相关联的打卡记录。

- 设备心跳(见 3.2.1)

- 服务端应答

```
-- HTTP header --  
response_code:OK  
trans_id:100  
cmd_code: DELETE_USER
```

```
-- HTTP body --  
{  
  "usersCount":100,  
  "userId":["1","2","3"...]  
}
```

- 设备应答

```
-- HTTP header --  
request_code: send_cmd_result  
trans_id:100  
cmd_return_code:OK
```

```
-- HTTP body --  
{  
  "userId":["1","2","3"...]  
}
```

- 服务端确认(详见 2.8)

```
-- HTTP header --  
response_code:OK  
trans_id:100
```

● 参数解释

字段	必须项	说明
usersCount	是	想要删除的用户数量，大小应和 users_id 数组大小一致。 若数量为 0 表示想要删除所有数据， users_id 数组可省略。
usersId	否	服务端下发的表示想要删除的用户 ID 数组。客户端应答的表示删除失败的用户 ID 数组

3.2.7 GET_LOG_DATA (获取记录数据)

从机器获取某一个时间段之内的机器记录数据

- 设备心跳(见 3.2.1)

- 服务端应答

```
-- HTTP header --  
response_code:OK  
trans_id:100  
cmd_code: GET_LOG_DATA
```

```
-- HTTP body --(beginTime 和 endTime 都可为空，但是 body 项不可为空，即可以为一个空 json 对象{})  
{  
  "newLog":0,"beginTime":"yyyyMMdd","endTime":" yyyyMMdd","clearMark":0  
}
```

- 设备应答

```
-- HTTP header --  
request_code: send_cmd_result  
trans_id:100  
cmd_return_code:OK
```

```
-- HTTP body --
```



```
{
  "packagelId":0,
  "allLogCount":26,
  "logsCount":26
  "logs":[{"
    "userId":"1",
    "time":"yyyyMMddHHmmss",
    "verifyMode":"Card+Face",
    "ioMode":1,
    "inOut":"In",
    "doorMode":"hand_open",
    "temperature":37.1,
    "logPhoto":"Base64"
  },...]
}
```

● 服务端确认(详见 2.8)

```
-- HTTP header --
response_code:OK
trans_id:100
```

● 重复设备应答和服务端确认流程直到 package_id 的值为 0 即数据传输完成

● 参数解释

字段	必须项	说明
newLog	否	是否要采取新纪录，1:是，将忽略时间段并且清理记录标志，0:否，默认为 0

beginTime	否	起始时间，若不存在本字段表示所有结束时间之前的数据
endTime	否	结束时间，若不存在本字段表示所有起始时间之后的数据
clearMark	否	是否清理记录该时间段内的记录标志，0:否，1:是，默认为 0
packageId	是	数据包 ID，需要上传的数据过大的情况下，设备将会对数据进行分包传输，数据包 ID 从 1 开始计数，依次递增，0 表示数据全部传输完成
allLogsCount	是	所有需要上传的记录数量。
logsCount	是	本包数据中包含的记录数量
Logs	是	设备应答的用户数组。
userId	是	打卡用户 ID
Time	是	打卡时间
verifyMode	是	打卡方式，指纹、人脸等
ioMode	否	打卡模式
inOut	否	进出模式
doorMode	否	开门模式
temperature	否	打卡温度
logPhoto	否	打卡照片，二进制数据，base64 格式

3.2.8 CLEAR_LOG_DATA (删除所有记录数据)

删除设备上所有的打卡记录

- 设备心跳(见 3.2.1)

- 服务端应答

```
-- HTTP header --  
response_code:OK  
trans_id:100  
cmd_code: CLEAR_LOG_DATA
```

- 设备应答

```
-- HTTP header --  
request_code: send_cmd_result  
trans_id:100  
cmd_return_code:OK
```

- 服务端确认(详见 2.8)

```
-- HTTP header --  
response_code:OK  
trans_id:100
```

- 参数解释

本指令没有参数

3.2.9 SET_TIME (同步时间)

同步设备时间。

- 设备心跳(见 3.2.1)

- 服务端应答

```
-- HTTP header --  
response_code:OK  
trans_id:100  
cmd_code: SET_TIME
```

```
-- HTTP body --  
{  
  "syncTime":"yyyyMMddHHmmss"  
}
```

- 设备应答

```
-- HTTP header --  
request_code: send_cmd_result  
trans_id:100  
cmd_return_code:OK
```

- 服务端确认(详见 2.8)

```
-- HTTP header --  
response_code:OK  
trans_id:100
```

- 参数解释

字段	必须项	说明
syncTime	是	服务器想要同步的时间

3.2.10 RESET_FK (机器重新启动)

在由于某些原因要重启考勤机的时候发送这个指令。
重启以后机器将忽视以前执行的所有的指令而进入接收新指令的状态。
本指令没有设备应答和服务端确认流程，不带任何参数

- 设备心跳(见 3.2.1)

- 服务端应答

```
-- HTTP header --  
response_code:OK  
trans_id:100  
cmd_code: SET_TIME
```

3.2.11 UPDATE_FIRMWARE (更新固件)

通知设备更新固件，注意设备升级时千万不要断电。

- 设备心跳(见 3.2.1)
- 服务端应答 (注意这里的 key 值 “firmwareFileURL” 因为历史错误原因使用的大写的 “i” 和小写的 “L”)

-- HTTP header --

response_code:OK

trans_id:100

cmd_code: UPDATE_FIRMWARE

-- HTTP body --

{

 "firmwareFileURL":"www.xxx.com/xxxx.bin"

}

- 设备应答

该指令设备不作更多的应答，服务端收到设备对下发 URL 请求便说明设备收到服务器的指令正在进行固件下载更新

- 参数解释

字段	必须项	说明
firmwareFileURL	是	固件的 URL 地址，设备收到此 URL 之后将会停止心跳包并向目标地址下载新的固件，更新完成后自动重启。注意设备应答只能说明收到 URL，不能说明升级成功，服务端可以通过 GET_DEVICE_INFO 指令查询固件版本判断设备是否升级成功

3.2.12 CLEAR_MANAGER(清除设备管理员)

清除设备内的管理员标志，并不会删除管理员的用户信息

- 设备心跳(见 3.2.1)
- 服务端应答
 - HTTP header --
 - response_code:OK
 - trans_id:100
 - cmd_code: CLEAR_MANAGER
- 设备应答
 - HTTP header --
 - request_code: send_cmd_result
 - trans_id:100
 - cmd_return_code:OK
- 服务端确认(详见 2.8)
 - HTTP header --
 - response_code:OK
 - trans_id:100
- 参数解释

本命令无参数

3.2.13 ENTER_ENROLL (远程进入注册界面)

通过下发指令进入注册界面

- 设备心跳(见 3.2.1)
- 服务端应答
 - HTTP header --
response_code:OK
trans_id:100
cmd_code: ENTER_ENROLL
 - HTTP body --
{
 "userId":"666",
 "feature":"face"
}
- 设备应答
 - HTTP header --
request_code: send_cmd_result
trans_id:100
cmd_return_code:OK

- 服务端确认(详见 2.8)

-- HTTP header --
response_code:OK
trans_id:100

- 参数解释

字段	必须项	说明
userId	是	要注册的用户 ID
feature	是	要注册的特征类型, face:人脸、fp:指纹、card:卡、pwd:密码

3.2.14 SET_DEVICE_SETTING(设置设备参数)

下发设备的各种设置参数

- 设备心跳(见 3.2.1)

- 服务端应答

-- HTTP header --
response_code:OK
trans_id:100
cmd_code: SET_DEVICE_SETTING

-- HTTP body --(具体的设置表见参数列表)

```
{
  "xxx":"xxx",
  "xxxx":{
    "xx":"xx"
    ...
  }
  ...
}
```

● 设备应答

```
-- HTTP header --
request_code: send_cmd_result
trans_id:100
cmd_return_code:OK
```

● 服务端确认(详见 2.8)

```
-- HTTP header --
response_code:OK
trans_id:100
```

● 参数解释(本指令所有字段都非必须项,最终的 value 值都为 string)

字段	说明
devName	设备名
wiegandType	维根格式(26/34)
serverHost	服务器地址，注意本项修改后轮询地址被改变，原服务端将收不到接下来的指令

serverPort	服务器端口，注意本项修改后轮询端口被改变，原服务端将收不到接下来的指令	
pushServerHost	推送服务器地址	
pushServerPort	推送服务器端口	
pushEnable	是否开启实时推送功能(yes/no)	
interval	心跳间隔(秒)	
language	设备语言(根据固件类型可选，例如 CHS/CHT/en/es/es-AR 等，详见附录)	
volume	设备音量(1~10)	
antiPass	是否开启反潜回(yes/no)	
openDoorDelay	开门延时(秒)	
tamperAlarm	是否启用防拆报警(yes/no)	
alarmDelay	报警延时(秒)	
reverifyTime	重复确认时间(分钟)	
screensaversTime	主界面无变化后多久时间进入屏保模式(秒),0 表示关闭屏保	
sleepTime	主界面无变化后多久时间进入睡眠模式(分钟),0 表示关闭睡眠	
verifyMode	打卡方式选择(可选列表见附录)	
living	是否开启活体功能(no/yes/double)或者(lv0:关闭活体;lv1:中级活体;lv2:高级活体)	
inOut	门禁机状态(in/out)	
MultiCheck	同时确认数(1~10)	
distance	识别距离(near/mid/far)	
matchThreshold	识别阈值(10~80)	
liveThreshold	识别阈值(10~80)	
adminPwd	oldPwd	设置 webserver 密码，oldPwd 表示之前的密码，newPwd 为新密码
	newPwd	
brightLed	mode	补光灯模式；on：常开；off：常闭；auto：自动；timing：定时开关，时间设置如下 定时开启时间，格式为 HHmm
	start	

	end	定时关闭时间，格式为 HHmm
--	-----	-----------------

3.2.15 GET_DEVICE_SETTING(获取设备参数)

根据需求获取设备内的各种设置参数

- 设备心跳(见 3.2.1)

- 服务端应答

-- HTTP header --

response_code:OK

trans_id:100

cmd_code: GET_DEVICE_SETTING

- 设备应答

-- HTTP header --

request_code: send_cmd_result

trans_id:100

cmd_return_code:OK

-- HTTP body --(返回的参数列表同 SET_DEVICE_SETTING)

{

 "xxx":"xxx",

 "xxx":{

 "xx":"xx"

 ...

```
}  
...  
}
```

- 服务端确认(详见 2.8)

```
-- HTTP header --  
response_code:OK  
trans_id:100
```

- 参数解释

同 SET_DEVICE_SETTING

3.2.16 RESET_DEVICE (恢复出厂设置)

恢复出厂时的大部分设置参数

- 设备心跳(见 3.2.1)
- 服务端应答
 - HTTP header --
 - response_code:OK
 - trans_id:100
 - cmd_code: RESET_DEVICE
- 设备应答
 - HTTP header --
 - request_code: send_cmd_result
 - trans_id:100
 - cmd_return_code:OK
- 服务端确认(详见 2.8)
 - HTTP header --
 - response_code:OK
 - trans_id:100
- 参数解释
 - 本指令没有参数

3.2.17 SET_DOOR_STATUS (设置门控继电器状态)

设置门控继电器状态从而控制开关门

- 设备心跳(见 3.2.1)

- 服务端应答

-- HTTP header --

response_code:OK

trans_id:100

cmd_code: SET_DOOR_STATUS

-- HTTP body --

```
{  
  "doorStatus":"open_close"  
}
```

- 设备应答

-- HTTP header --

request_code: send_cmd_result

trans_id:100

cmd_return_code:OK

- 服务端确认(详见 2.8)

-- HTTP header --

response_code:OK

trans_id:100

- 参数解释

字段	必须项	说明
doorStatus	是	close:关; open:开(不会自动关闭); open_close:开, 经过延时时间后自动关。

3.2.18 RESET_ENROLL_MARK (重置实时登记数据标志)

通过下发指令清空实时登记数据标志

- 设备心跳(见 3.2.1)

- 服务端应答

-- HTTP header --

response_code:OK

trans_id:100

cmd_code: RESET_ENROLL_MARK

- 设备应答

-- HTTP header --

request_code: send_cmd_result

trans_id:100

cmd_return_code:OK

- 服务端确认(详见 2.8)

```
-- HTTP header --  
response_code:OK  
trans_id:100
```

3.2.19 RESET_LOG_MARK (重置实时打卡数据标志)

通过下发指令清空某时间段内实时打卡数据标志

- 设备心跳(见 3.2.1)

- 服务端应答

```
-- HTTP header --  
response_code:OK  
trans_id:100  
cmd_code: RESET_LOG_MARK
```

```
-- HTTP body --(beginTime 和 endTime 都可为空，但是 body 项不可为空，即可以为一个空 json 对象{})
```

```
{  
  "beginTime":"yyyyMMdd","endTime":" yyyyMMdd"  
}
```

- 设备应答

```
-- HTTP header --  
request_code: send_cmd_result  
trans_id:100
```

cmd_return_code:OK

- 服务端确认(详见 2.8)

-- HTTP header --

response_code:OK

trans_id:100

- 参数解释

字段	必须项	说明
beginTime	否	起始时间，若不存在本字段表示所有结束时间之前的数据
endTime	否	结束时间，若不存在本字段表示所有起始时间之后的数据

3.2.20 SET_USERS_TIME_GROUPS (设置用户门禁时间段)

通过下发指令设置用户的门禁时间段

- 设备心跳(见 3.2.1)

- 服务端应答

-- HTTP header --

response_code:OK

trans_id:100

cmd_code: SET_USERS_TIME_GROUPS

-- HTTP body --

```
{
  "userId":["1","2","3...],
  "timeGroups":{
    "sun":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
    "mon":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
    "tue":["HHmmHHmm","HHmmHHmm","HHmmHHmm","HHmmHHmm"],
    "wed":["HHmmHHmm","HHmmHHmm","HHmmHHmm"],
    "thu":["HHmmHHmm","HHmmHHmm"],
    "fri":["HHmmHHmm"]
  }
}
```

- 设备应答

```
-- HTTP header --
request_code: send_cmd_result
trans_id:100
cmd_return_code:OK

-- HTTP body --
{
  "userId":["1","2","3...]
}
```

- 服务端确认(详见 2.8)

```
-- HTTP header --
response_code:OK
```

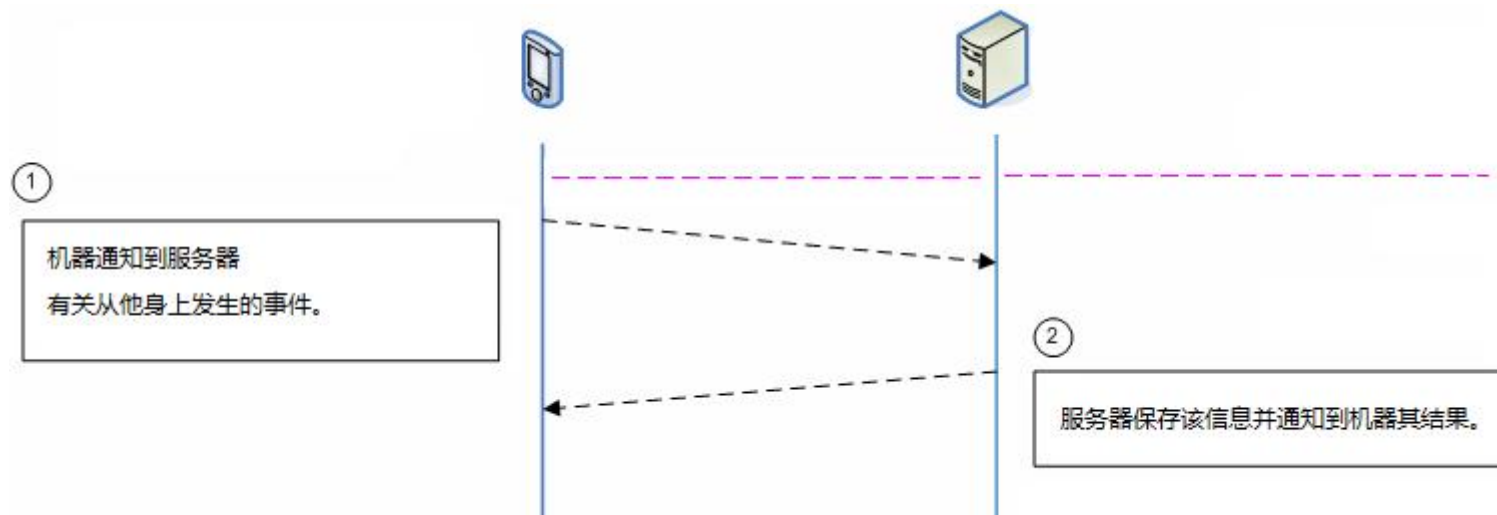
trans_id:100

● 参数解释

字段	必须项	说明
userId	否	想要设置的用户 ID 数组，若此项不存在，则视为设置设备内所有用户的门禁数据。
timeGroups	是	要设置的门禁时间段
sun	否	周日时段,最多可有 6 个字符串表示时段（起始时分到结束时分），数组长度可以小于 6 个，缺少的默认取"00000000"，本字段不存在表示全部取"00000000"，即本日不允许通过，全天可用可填["00002359"]
mon	否	周一时段，其他同上
tue	否	周二时段，其他同上
wed	否	周三时段，其他同上
thu	否	周四时段，其他同上
fri	否	周五时段，其他同上
sat	否	周六时段，其他同上

4 . 实时数据传输功能

实时传输功能是指当设备某些时间发生时及时通知 WEB 服务器，实时传输是考勤机主动向 WEB 服务器发送请求接收应答。其中包括实时记录传输功能。



4.1 实时打卡记录传输

实时记录传输是指在机器上有新记录产生时主动推送到服务器的功能。

- 设备推送

```
-- HTTP header --
```

```
request_code: realtime_glog
```

```
-- HTTP body --
```

```
{  
    "userId":"1",  
    "time":"yyyyMMddHHmmss",  
    "verifyMode":"Card+Face",  
    "ioMode":1,  
    "inOut":"In",  
    "doorMode":"hand_open",  
    "temperature":37.1,  
    "logPhoto":"Base64"  
}
```

- 服务端确认(详见 2.8)

```
-- HTTP header --
```

```
response_code:OK
```

trans_id:100

- 参数解释

本指令推送的打卡记录固定为每次一条，参数同 3.2.7

4.2 实时门状态

实时门状态是指设备检测门磁的状态实时传送至服务器。

- 设备推送

-- HTTP header --

request_code: realtime_door_status

-- HTTP body --

```
{  
    "doorStatus":"open"  
}
```

- 服务端确认(详见 2.8)

-- HTTP header --

response_code:OK

trans_id:100

- 参数解释

字段	必须项	说明
doorStatus	是	门磁状态, open:开, close:关

4.3 实时登记数据传输

实时登记数据传输是是每当在设备上更新登记数据时机器发送该数据到服务器的功能, 服务端下发的数据不会触发此推送

- 设备推送

-- HTTP header --

request_code: realtime_enroll_data

-- HTTP body --

```
{
  "userId":"1",
  "name":"张三",
  "privilege":0
  "card":"123456"
  "pwd":"666"
  "fps":["base64_0"," base64_1",...," base64_9"],
  "face":"base64",
  "palm":["base64_0"," base64_1"]
  "photo":"base64"
  "vaildStart":"yyyyMMddHHmmss"
```

```
    "vaildEnd": "yyyyMMddHHmmss"  
  }
```

- 服务端确认(详见 2.8)

```
-- HTTP header --  
response_code:OK  
trans_id:100
```

- 参数解释

本指令推送的用户数据固定为每次一条，参数同 3.2.5

5 . 版本更新记录

更新日期	更新说明	备注

--	--	--